

SQL Server 性能优化



高效索引指南

舒永春

2018-10-11

性能不够，索引来凑

- 性能不好，建个索引就会好？
- 索引一定能提升性能？

躺枪了.....怪我咯



高效索引指南

- 提高索引的存储效率
- 选择合适的索引
- 减少二次查找
-



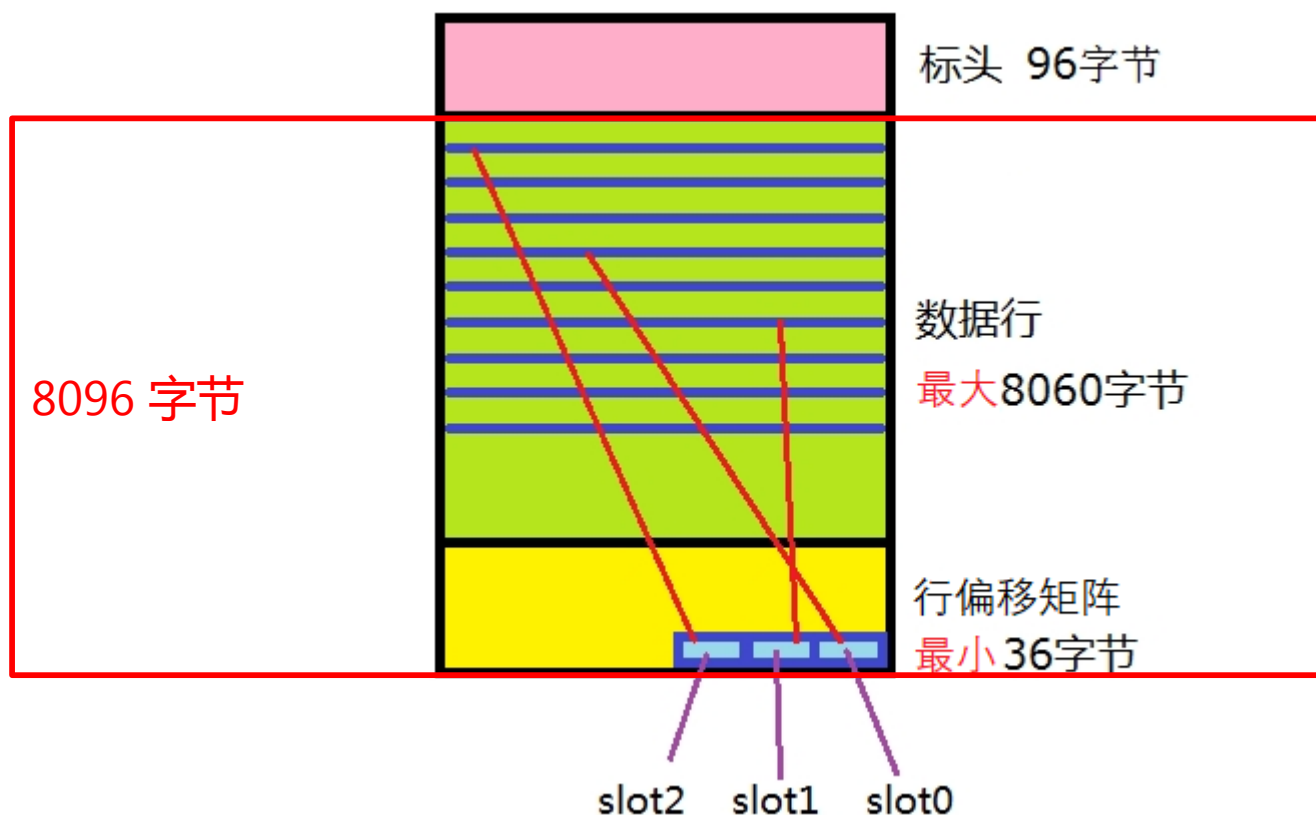
已经准备好饮料零食西瓜鸡腿坐等开播

第一式

提高存储效率

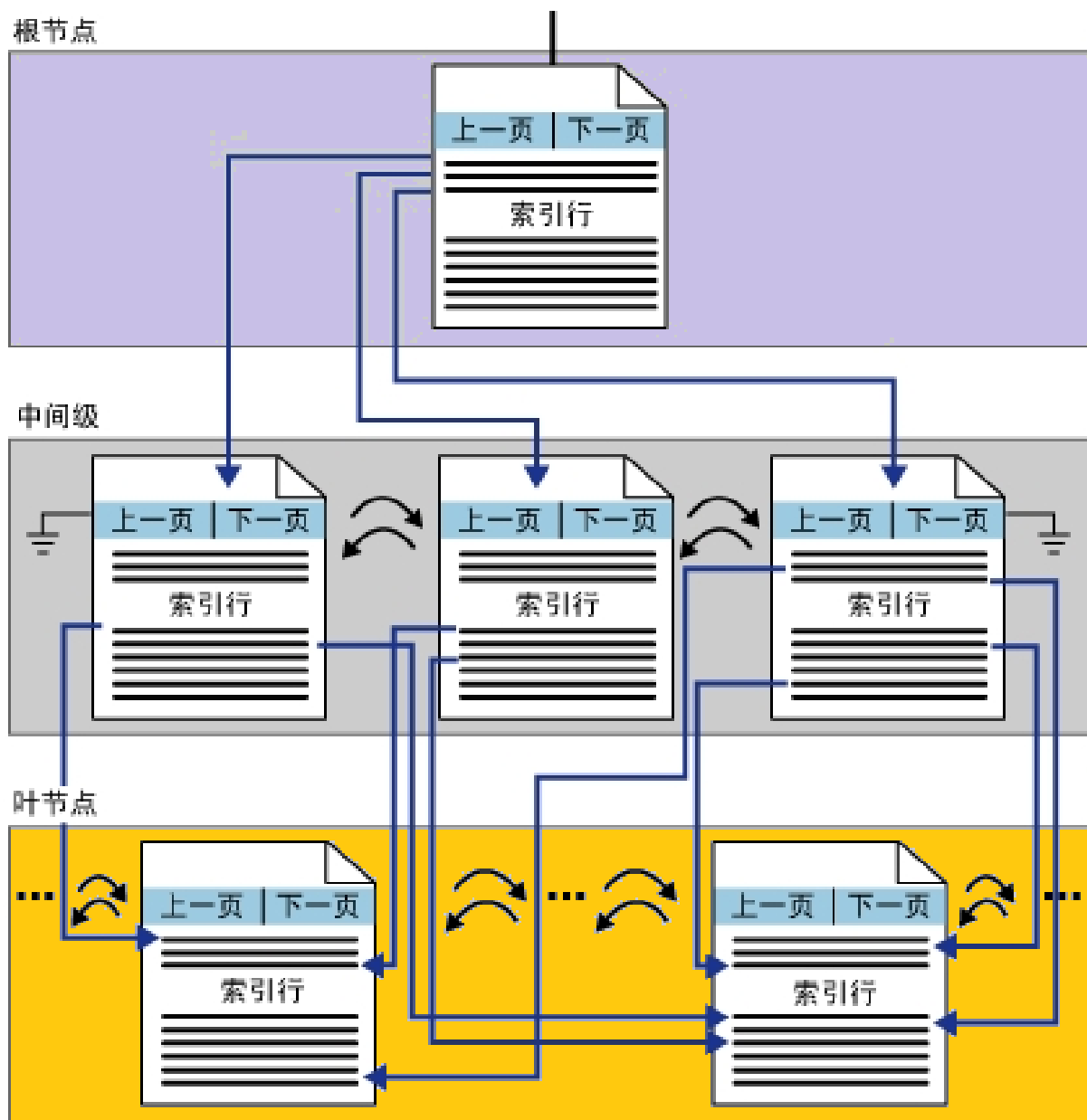
索引页的结构

- 8 KB 页面
 - 标头 96 字节
 - 数据行 + 行偏移矩阵 = 8060 字节



索引的层次结构

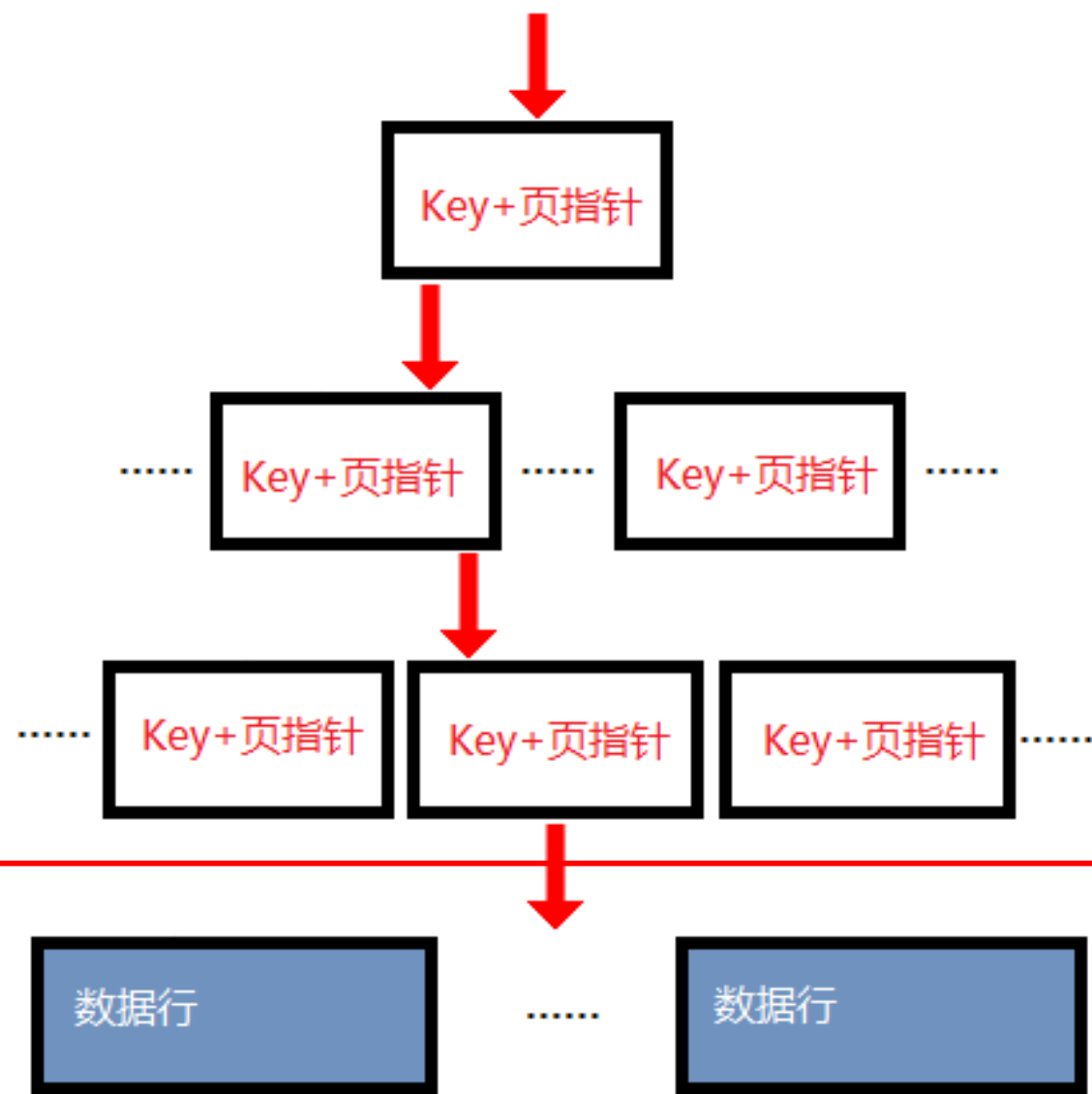
- 非叶级
 - 根节点
 - 中间级
- 叶级



非叶级与叶级的数据格式

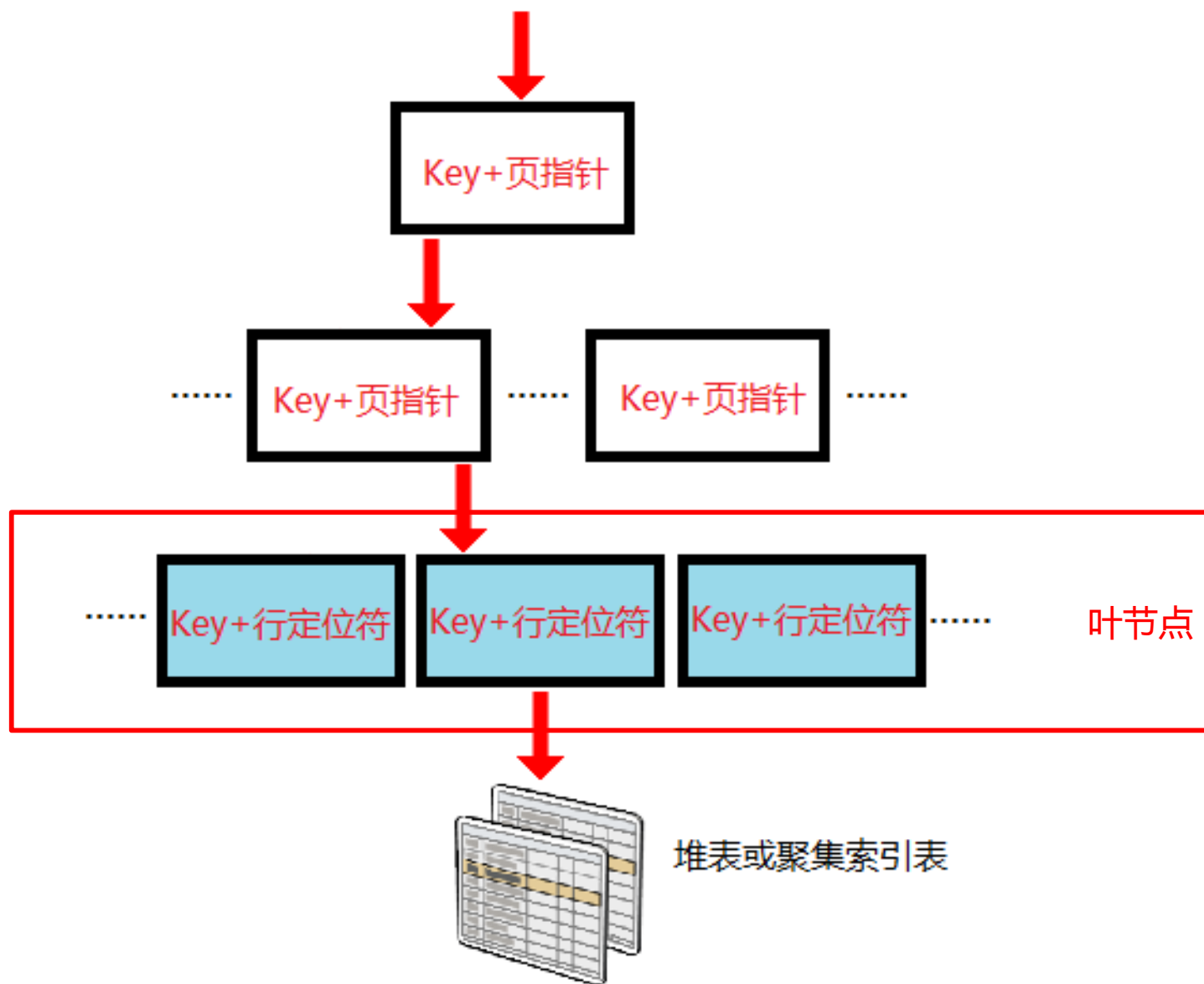
- 非叶级
 - Key + 下一级页面指针
- 叶级
 - 聚集索引：数据行
 - 非聚集索引：Key + 行定位符

聚集索引



假如非叶级每层的索引条目
宽度均为 40 字节，
假如中间级有 2 层，
最多可以管理的数据行约为
 $(8000/40)^3 = 800$ 万行

非聚集索引



非聚集索引 (index_id>1) 叶级的行定位符

- 底层是堆表 (index_id=0)
 - 行定位符是 RID , 直接指向堆上的数据行
 - 包括 : file_id, page_id, slot_id
 - 8 字节
- 底层是聚集索引表 (index_id=1)
 - 行定位符是聚集索引的 Key
 - 缺点 : 可能发生二次查找

如果索引不是 Unique

- 聚集索引
 - 隐含了 4 字节的 uniqueifier 列
- 堆上的非聚集索引
 - 包含行定位符 (RID)
- 聚集索引上的非聚集索引
 - 组合非聚集索引与聚集索引的键列，并从聚集索引的键列中去除重复项。
 - 如果聚集索引不是 Unique，则同时组合聚集索引隐含的 uniqueifier 列。

权衡

- 堆表 vs. 聚集索引表
 - 大量读 vs. 大量更新
 - 精确查找的成本
 - 索引的维护成本（碎片、重整、锁）
 - 其他因素
- Unique 索引 vs. 非 Unique 索引

索引碎片整理

- 增删改导致索引页面出现碎片
 - 新页面跟原始页面可能不连续
 - 拆分时尽量保证均分：中间级和叶级页被拆分，原始页面留下一半的行，另一半被转移到新页
 - 保证根页总是1页：根页如果需要被拆分，则会新分配2个页插入一个中间级，新的根页只有2行
- 预留填充空间
 - 填充因子（ FILLFACTOR ）
- 整理
 - 重整（ REORGANIZE ）：联机对叶级页物理排序
 - 重建（ REBUILD ）

设计合适的数据类型

- date vs. datetime
- int vs. char
- Uniqueidentifier vs. IDENTITY 列
- varchar(max) / nvarchar(max) ?



仅选择必要的列

- 索引的最大列数
 - 旧版本最多 16 列，SQL Server 2016 增加到 32 列
- 索引的最大字节数
 - 聚集索引 900 字节，非聚集索引 1700 字节
 - 如果包含 varchar/nvarchar 且超过上述限制，创建索引时会有警告信息；INSERT/UPDATE 时如果超限则操作失败。



消息

警告！最大键长度为 900 个字节。索引 'idx_Bar' 的最大长度为 8000 个字节。对于某些大值组合，插入/更新操作将失败。

100 % <



查询已成功执行。



消息

消息 1946，级别 16，状态 3，第 2 行
操作失败。索引 'idx_Bar' 的索引条目长度为 901 字节，超出了允许的最大长度 900 字节。

100 % <

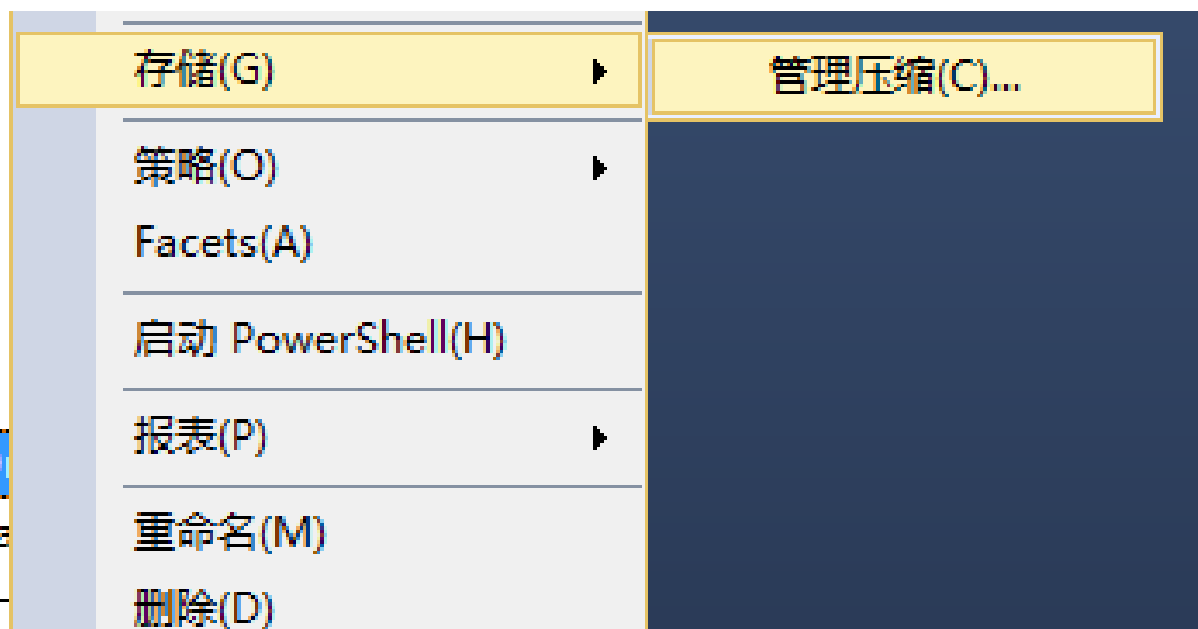
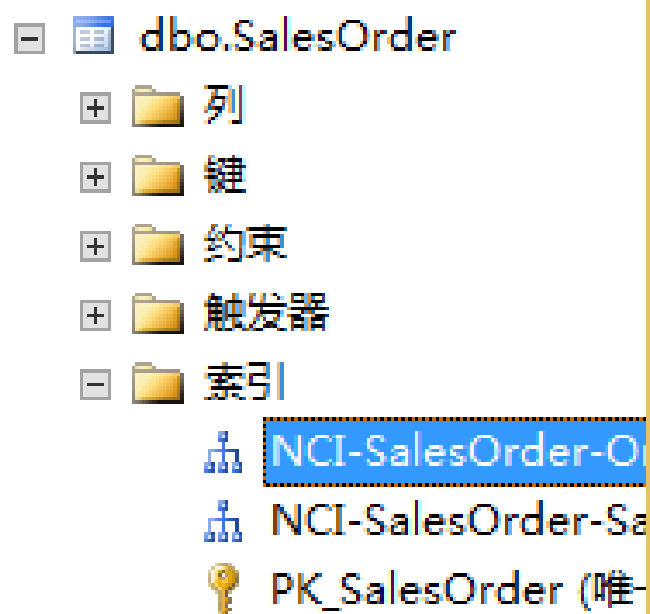


查询已完成，但有错误。

数据压缩

- 特点
 - 提高了页面存储效率
 - 少量增加了 CPU 的负荷
 - 企业版
- 实现方法 — 脚本
 - CREATE INDEX 和 ALTER INDEX 语句中
 - 指定压缩选项，例如 WITH DATA_COMPRESSION=PAGE
 - 三种压缩选项任选其一：NONE、ROW、PAGE

- 实现方法 — 图形界面
 - 在图形界面创建索引时，不提供压缩选项
 - 右键菜单“存储” - “管理压缩”
 - 导致两次操作：先创建索引，再修改索引



列存储索引

- SQL Server 2012 引入
 - 非聚集列存储索引
 - 只读
- SQL Server 2014 改进
 - 可更新聚集列存储索引
 - 数据暂存在“增量存储”中
- 列存储的特点
 - 极高的压缩率
 - 针对少数列的查询，效率高
 - 不适合精确查找行

第二式

减少维护的开销

合适的索引数量

- 查询速度 vs. 索引更新的开销
 - 例如：一个表有 36 个索引，如果新增 1 行数据，维护索引需要多大的开销？
- 大量Read vs. 大量Write
- 技术限制：每个表最多 999 个非聚集索引

锁的开销

- SELECT : S 锁、IS锁
- UPDATE :
 - U 锁 : 用到的索引
 - IU锁 : 查询涉及的页面
 - IX锁 : 修改涉及的页面
 - X 锁
 - 需要被修改的数据行
 - 需要被修改的索引键值。该索引使用了双倍的锁 (删除旧键值、插入新键值)

演示：UPDATE 需要的锁

详细脚本及解释 <http://www.mssqlmct.cn/t-sql/?post=75>



-- 1. 直接在堆表上执行 UPDATE

结果		消息				
	resource_type	object_name	request_mode	resource_description	resource_associated_entity_id	index_id
1	DATABASE	NULL	S		0	NULL
2	OBJECT	SalesOrderDetail	IX		1938105945	NULL
3	PAGE	NULL	IV	1:5297	72057594044547072	0
4	PAGE	NULL	IV	1:5302	72057594044547072	0
5	PAGE	NULL	IV	1:5303	72057594044547072	0
6	PAGE	NULL	IV	1:5300	72057594044547072	0
7	PAGE	NULL	IV	1:5301	72057594044547072	0
1508	PAGE	NULL	IV	1:6593	72057594044547072	0
1509	PAGE	NULL	IV	1:6592	72057594044547072	0
1510	PAGE	NULL	IV	1:6596	72057594044547072	0
1511	RID	NULL	X	1:6283:79	72057594044547072	0

表扫描

修改一行

-- 2. 创建非聚集索引再执行 UPDATE

结果 消息						
	resource_type	object_name	request_mode	resource_description	resource_associated_entity_id	index_id
1	DATABASE	NULL	S		0	NULL
2	KEY	NULL	U	(1fc8e159f3f4)	72057594044678144	4 RID lookup
3	OBJECT	SalesOrderDetail	IX		1938105945	NULL
4	PAGE	NULL	IX	1:6283	72057594044547072	0
5	PAGE	NULL	IU	1:7907	72057594044678144	4
6	RID	NULL	X	1:6283:79	72057594044547072	0 修改一行

-- 3. 创建聚集索引和非聚集索引再执行 UPDATE

结果		消息					
	resource_type	object_name	request_mode	resource_description	resource_associated_entity_id	index_id	
1	DATABASE	NULL	S		0	NULL	
2	KEY	NULL	X	(5f36df65313f)	72057594044743680	1	修改一行
3	KEY	NULL	X	(48b349c673cb)	72057594044809216	4	
4	KEY	NULL	X	(8c2f28ff3598)	72057594044809216	4	删+增
5	OBJECT	SalesOrderDetail	IX		1938105945	NULL	
6	PAGE	NULL	IX	1:9267	72057594044743680	1	
7	PAGE	NULL	IX	1:2207	72057594044809216	4	
8	PAGE	NULL	IX	1:2233	72057594044809216	4	

统计信息

- 表和视图
 - 当某列第一次最为条件查询时，将创建该列的统计信息（名称格式示例：_WA_Sys_00000002_44CA3770）
 - 当创建索引时，将创建同名的统计信息
- 只读数据库、快照、AlwaysOn 可用性组只读辅助副本
 - 在 tempdb 中创建和维护临时统计信息
 - 名称后缀 _readonly_database_statistic
 - sys_stats 视图包含一个 is_temporary 列
- 表变量、XML数据
 - 没有统计信息

[-] dbo.SalesOrder2012

+  列

+  键

+  约束

+  触发器

-  索引

 IX_SalesOrder2012_SalesOrderID (不唯一，非聚集)

-  统计信息

 _WA_Sys_00000002_276EDEB3

 _WA_Sys_00000004_276EDEB3

 _WA_Sys_00000005_276EDEB3

 _WA_Sys_00000006_276EDEB3

 IX_SalesOrder2012_SalesOrderID

自动更新统计信息

- 默认阈值
 - 总行数首次超过 500 行
 - 更新的行数超过 20%。例如，100 万行的表，更新 20 万行才触发
- 平滑更新
 - 使用跟踪标志 2371，针对超过 25000 行的表，启用平滑更新
 - SQL Server 2016（数据库的兼容性级别不低于 130）不需要跟踪标志 2371
 - 阈值改为： $1000 \times \sqrt{\text{当前的表基数}}$ 所得结果的平方根。例如，100 万行的表，阈值降到 3.2%

手动更新

- UPDATE STATISTICS
- 存储过程 sp_updatestats
- 注意：更新统计信息会导致查询重新编译

全表扫描与抽样

UPDATE STATISTICS ...

- WITH FULLSCAN
 - 全表扫描
- WITH SAMPLE (默认)
 - 默认由查询优化器决定抽样
 - 可以显式指定近似的百分比或行数
 - 可以显式 PERSIST_SAMPLE_PERCENT=ON , 保留设定的采样百分比
- WITH RESAMPLE
 - 使用最近的采样速率更新每个统计信息

索引视图

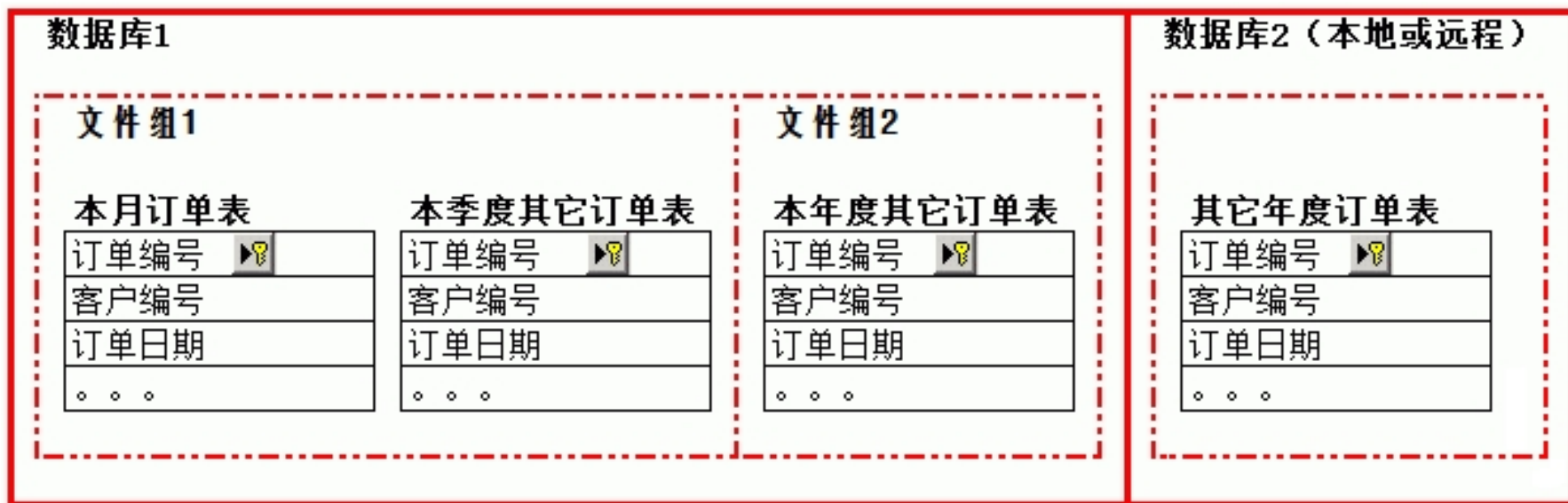
- “物化视图”
- 提升查询效率
- 第一个索引必须是 UNIQUE CLUSTERED INDEX , 然后可以创建非聚集索引
- 限制：
 - 只能引用基表，不能再嵌套其他视图
 - 必须是同一个数据库中的基表，表名必须为两部分名称
 - 必须 SCHEMABINDING
 - 不能包含 text、ntext、image
 - 不能使用 UNION

表和索引分区

- SQL Server 2005 引入
- 企业版
- 仍然还是一个表，只是将表中的数据水平分割并且分散储存到数据库的多个文件组
- 分区锁而不是表锁
- 并行索引

分区视图

- SQL Server 2000 的传统技术
- 特点：视图的每个基表使用 CHECK 约束
 - 查询优化器判断是否仅需访问指定的基表即可
 - 或者并行访问所有的基表



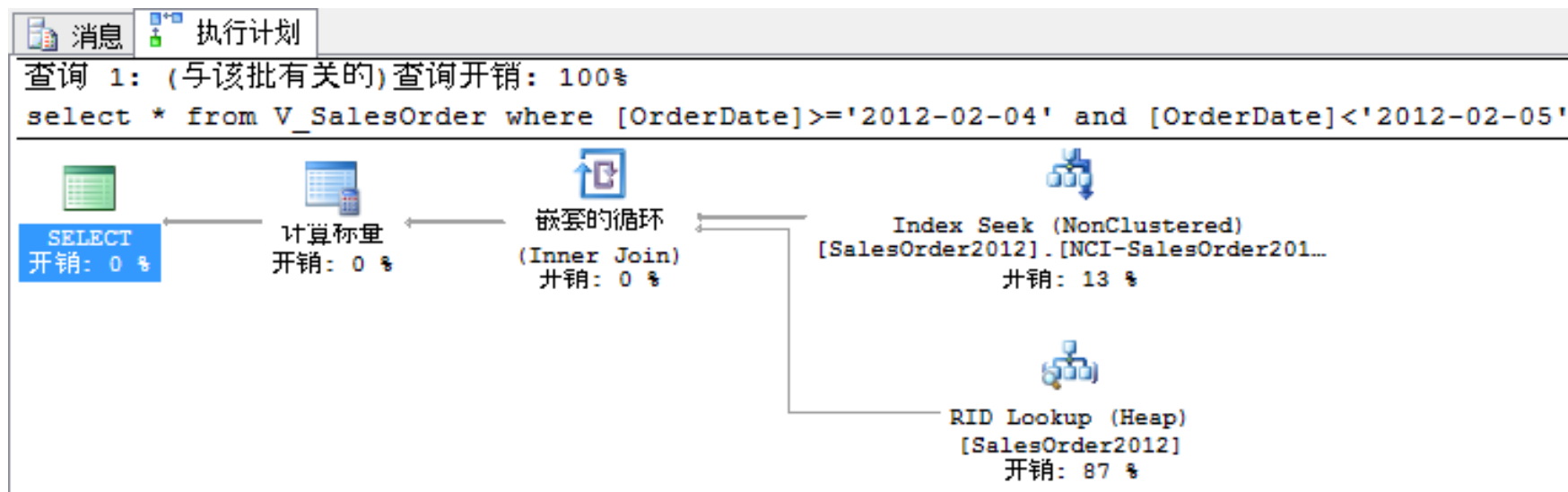
演示：分区视图

详细脚本及解释 <http://www.mssqlmct.cn/dba/?post=263>



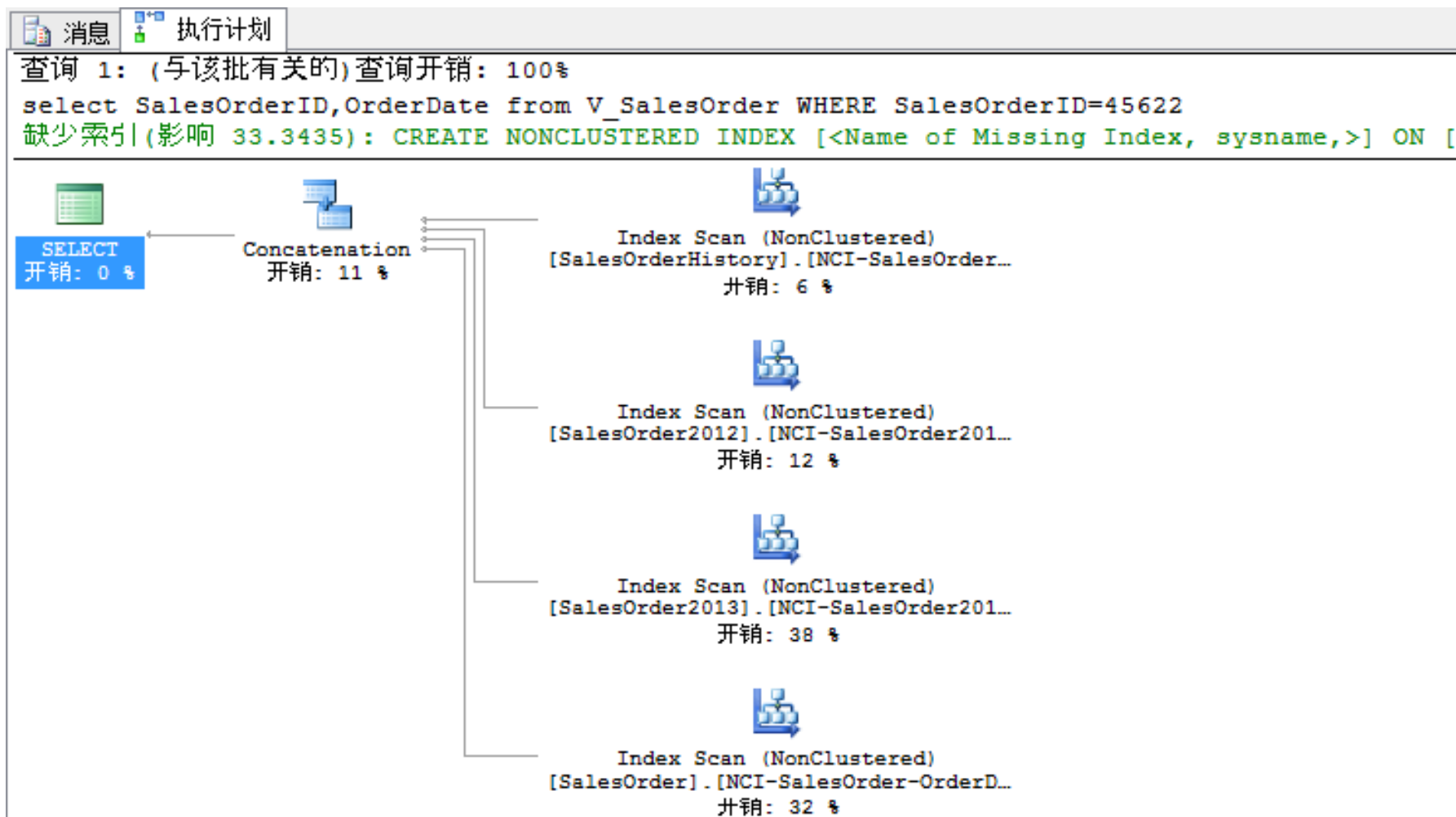
-- 查询特定日期之间的数据

```
select * from V_SalesOrder  
where [OrderDate]>='2012-02-04' and [OrderDate]<'2012-02-05'
```



-- 查询不特定日期的数据

```
select SalesOrderID, OrderDate from V_SalesOrder  
WHERE SalesOrderID=45622
```



选择合适的索引

基数估计

- 基数估计的算法版本
 - SQL Server 2014 之前是 70
 - SQL Server 2014 默认为 120
 - SQL Server 2016 默认为 130
 - SQL Server 2017 默认为 140
 - 可以强制使用旧的版本
- 预估的行数决定是否选择索引、选择哪个索引
- 估算失误，可能导致分配的内存不足，需要使用 tempdb，导致性能下降

统计信息直方图

- 统计信息只统计首列

- 索引除了按首列排序存储数据外，其统计信息也是按首列计算统计的，所以索引设置时定义的第一列非常重要。
- 每个统计信息对象都在包含一个或多个表列的列表上创建，并且包括显示值在第一列中的分布的直方图。

问：某工厂有数万名员工，其中男性不到 3%。为性别列创建了索引。以下查询是否会使用该索引？

```
SELECT * FROM 员工资料表  
WHERE 性别 = 女
```

筛选索引

- 是一种经过优化的非聚集索引
- 使用筛选谓词对表中的部分行创建索引
- 减少了索引维护开销

例1：某工厂的筛选索引“性别 = 男”，减少了无谓的存储空间

例2：某电商举办服装促销活动，为服装品类专门创建筛选索引

索引|覆盖

- 覆盖查询：索引包含了查询引用的所有列
- 如果索引需要引用更多的列
 - 降低了非叶级的存储效率
 - 索引的宽度和列数限制
 - 数据类型的限制
- 使用包含列
 - 使用 INCLUDE 子句
- 查询效率 vs. 索引维护开销

“确定的” 筛选条件

避免：

- WHERE ROUND ([dbo].[Amount], 0) > 100.0
- WHERE ISNULL([dbo].[PersonID], '') = 'F001'
- WHERE [dbo].[PersonID] LIKE '%F%'
-

Predicate

[tempdb].[dbo].[CityList].[CityName]
like 'F%'

对象

[tempdb].[dbo].[CityList].
[PK_CityList_CityName]

输出列表

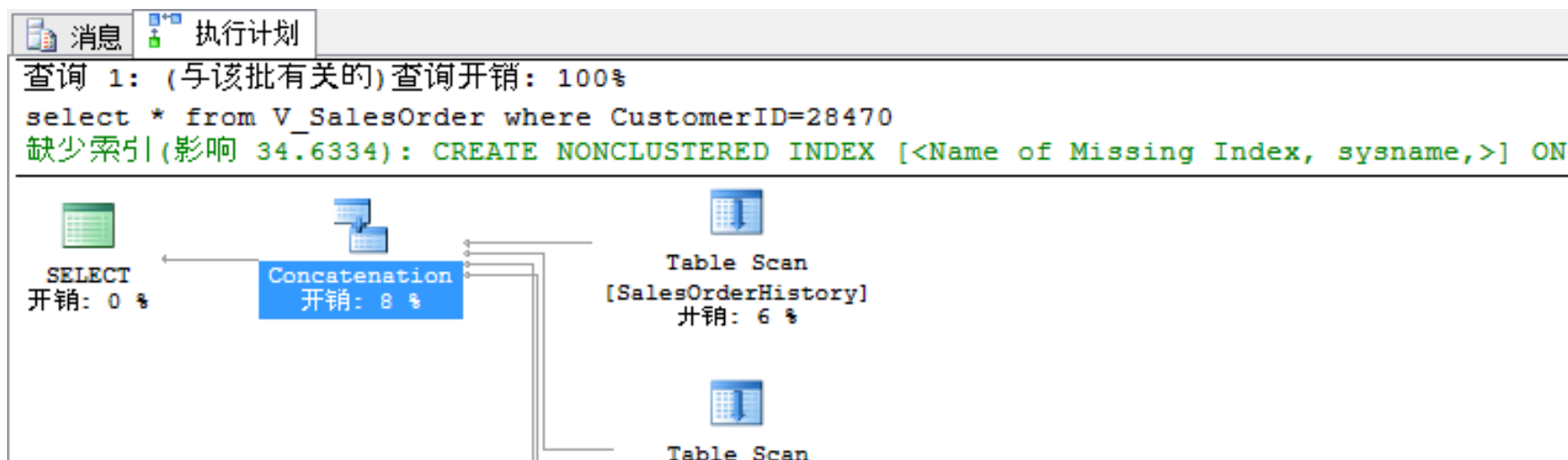
[tempdb].[dbo].[CityList].CityName,
[tempdb].[dbo].
[CityList].CityChineseName, [tempdb].
[dbo].[CityList].ProvinceID

Seek Predicates

Seek Keys[1]: Start: [tempdb].[dbo].
[CityList].CityName >= 标量运算符('F'),
End: [tempdb].[dbo].
[CityList].CityName < 标量运算符('G')

“缺失”的索引

- 查询执行计划提示创建索引能提高性能
- “缺失”的索引将记录在 sys.dm_db_missing_index_details 视图
- 重启后会重置



第四式

提高多表连接的效率

物理连接的方式

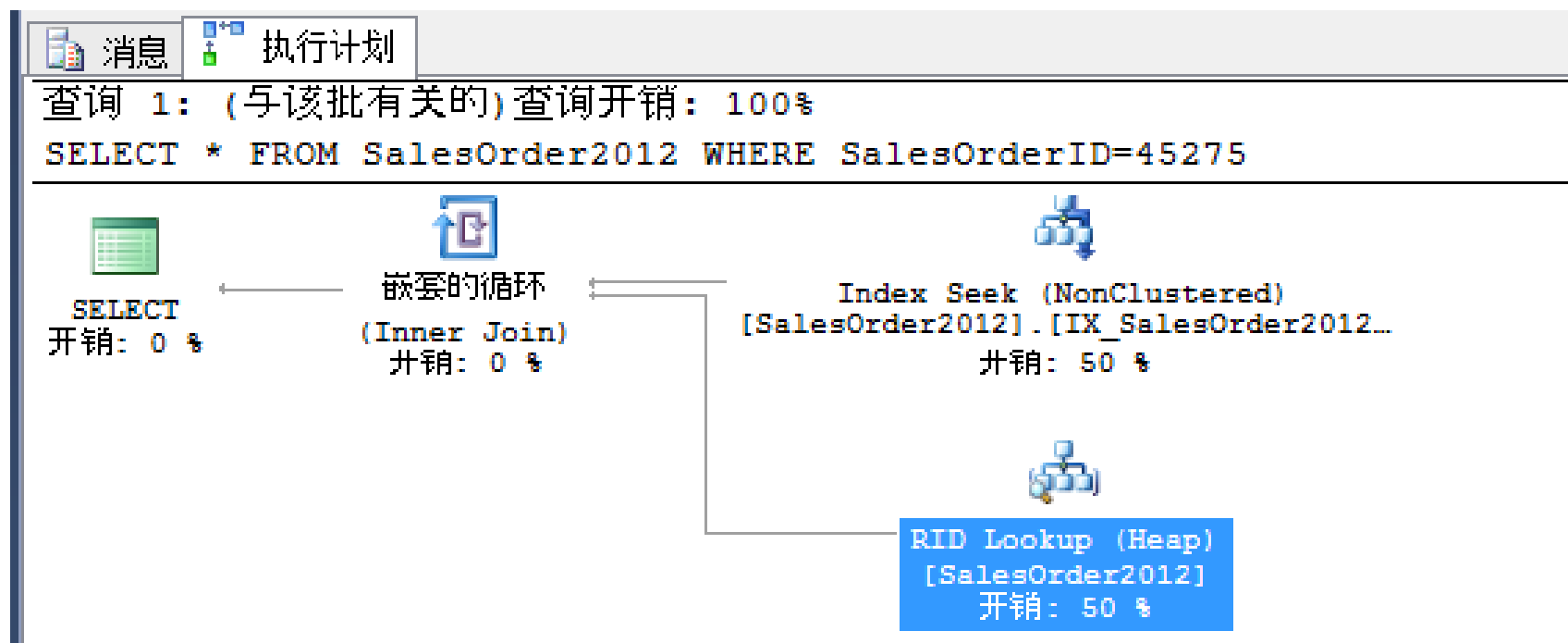
查询优化器自动选择最佳的连接方式，也可以显式指定连接方式

- Nested Loops
 - 第二个输入为第一个输入的每一个值去查找一次
 - 第二个输入的查找必须是低成本的
- Merge Join
 - 两个排序的输入是交叉的
 - 两个输入都必须是排序的
- Hash Match
 - 从第一个输入创建的一个哈希表跟第二个输入比较哈希值
 - 大量没有排序的输入

书签查找 – RID Lookup

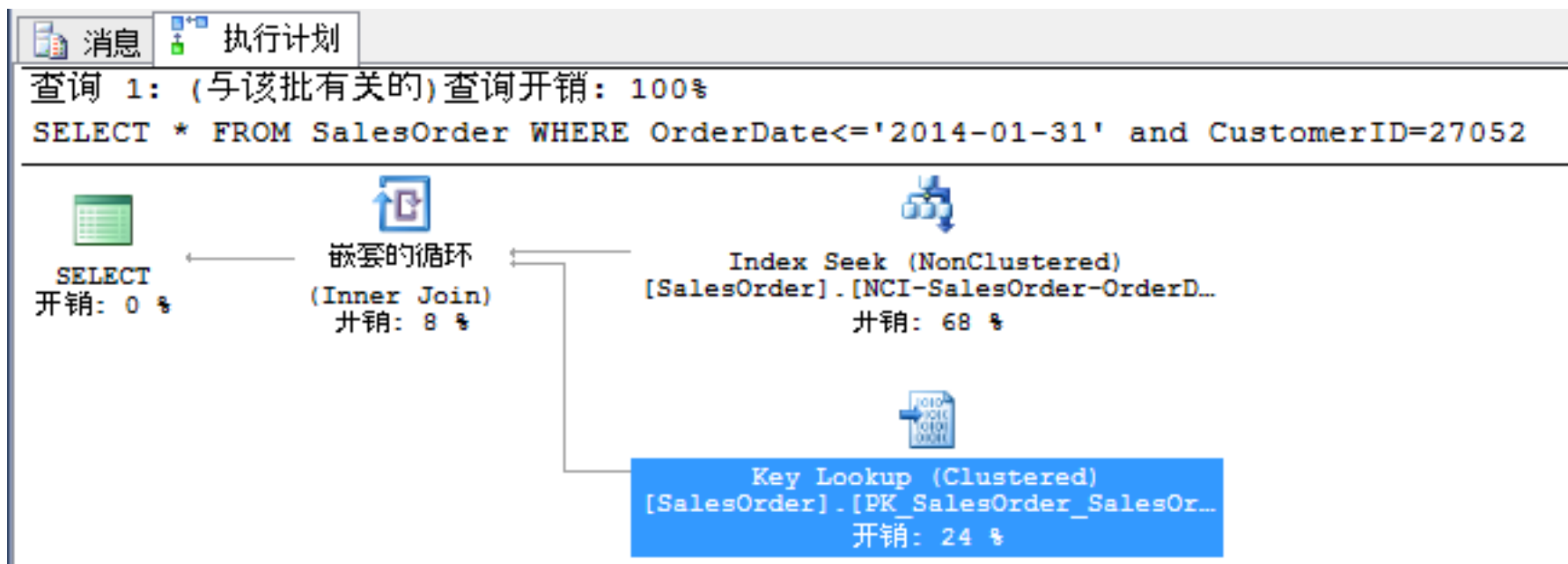
非聚集索引只包含查询的一部分的列，通过行定位符进行二次查找

- 堆表需要嵌套 RID 查找



书签查找 – Key Lookup

- 聚集索引表需要嵌套键查找



避免书签查找

- 尽量避免“SELECT * FROM”
- 索引覆盖
- 某些场景下，使用表扫描的效率更高
- 如果缺少的列在另一个索引上，可以索引交叉
 - SELECT...FROM 表 WITH(INDEX(索引1,索引2))
 - Hash Match

资源

- <https://books.goalkicker.com/MicrosoftSQLServerBook/>
- <https://www.sqlpassion.at/academy/performance-tuning-training-plan/>

下期课程：

SQL Server 临时表性能优化